F1.5 — Move authentication server-side with hashed passwords

Issue: oanyamele/flyerrr#16 **Owner:** `nadcaesar` (Nicholas Caesar) **Milestone:** Sprint 1 — Backend & Seam (Weeks 2–4) **Labels:** `backend` · `security` · `P0-blocking` **Estimate:** 5 story points **Traces to:** R-03 (plaintext passwords), R-05 (bypassable authorization)

---

1. Where this sits in the team's paperwork

F1.5 is the technical-debt remediation half of two items that already exist in the requirements traceability matrix in *Senior Project 1_ Group 1*:

Working-doc artifact	Relationship to F1.5
F7 — Dual Authentication Sign-In (`F07-02 Implement secure primary authentication` , owner NC, Sprint 1, P0, 5 pts)	F1.5 **is** F07-02 applied to the existing prototype. Same owner, same sprint, same estimate.
F07-03 — second authentication factor	The follow-on. F1.5 deliberately stops at the primary factor; do not pull 2FA into this issue.
F19 — Role-Based Access Control (owner OA)	F1.5 supplies the trustworthy identity that F19 authorizes against. F1.5 must define the session/role claim contract F19 consumes.
F21 — Attendance Data Store (owner NJ)	Supplies the `users` table F1.5 writes hashes into.
US-06 / US-07 / US-08 / US-09	All four are gated on an officer or admin role that cannot currently be trusted.

So when the professor asks "which requirement does this serve," the answer is: US-08 and US-07 are unprovable today because the role that protects them is a client-editable string. F1.5 is what makes them demonstrable.

---

2. Verified current state

Every claim below was read out of the working tree, not assumed.

2.1 Passwords are stored and compared in plaintext

```
password: string;
```

That field is serialized whole into `localStorage['flyer_db_v1']` by `writeDb` (`src/data/appDatabase.ts:108-118`), so every account password is readable from the DevTools → Application panel.

Two seed accounts ship with a literal password:

```
      id: "user-admin-1",
      email: "officer@howard.edu",
      password: "password123",
      // ...
      id: "user-student-1",
      email: "student@howard.edu",
      password: "password123",
```

Authentication is a string equality check with no hashing anywhere in the repo:

```
  const match = db.users.find(
    (user) =>
      user.email.toLowerCase() === normalizedEmail &&
      user.password === input.password &&
      user.role === input.role,
  );

  if (!match) return { ok: false as const, error: "Invalid email, password, or role." };
```

> Note for the issue body: the seed script `scripts/seed-backlog.sh` cites `appDatabase.ts:179` for this comparison. The real line is **187**. Correct it when you update the issue, and cite `docs/BACKLOG_INTEGRATION.md`, which already has it right.

2.2 The session is a client-writable object

```
  const [auth, setAuth] = useState<AuthState>(() => {
    const saved = localStorage.getItem("flyerrr_auth");
    return saved ? JSON.parse(saved) : { isLoggedIn: false, role: "student", userName: "" };
  });

  const login = (role: UserRole, name: string, options?: { email?: string; userId?: string }) => {
    const state = { isLoggedIn: true, role, userName: name, email: options?.email, userId:
options?.userId };
    setAuth(state);
    localStorage.setItem("flyerrr_auth", JSON.stringify(state));
  };
```

The only gate in front of every privileged screen reads that object:

```
const ProtectedRoutes = () => {
  const { auth } = useAuth();
  if (!auth.isLoggedIn) return <Navigate to="/login" replace />;
```

One console paste grants the verification queue, attendance reports, and event creation:

```
localStorage.setItem('flyerrr_auth', '{"isLoggedIn":true,"role":"org_officer"}')
```

2.3 A third hole the issue does not yet mention — add it

`role` is chosen by the user in the sign-up UI and passed straight into account creation. The officer/student choice is two buttons on the login screen (`src/pages/Login.tsx:66` and `:80`), and the selected value flows unvalidated into `createUser` :

```
    if (authMode === "signup") {
      if (!name.trim()) return;
      const result = createUser({ email, password, name, role });
```

Anyone can self-register as `org_officer` . That is a **privilege-escalation-by-signup** defect, and it is arguably worse than the localStorage edit because it requires no DevTools at all. Add it to issue #16 as a fourth piece of evidence and add a matching acceptance criterion (see §6).

2.4 There is no server to move anything to

`package.json` has zero server, database, hashing, or session dependencies — no Express/ Fastify/Hono, no `pg` , no `bcrypt` / `argon2` , no `jose` . The repository contains one Vite SPA. **F1.5 cannot be completed inside `src/` .**

---

3. The dependency problem — read this before you start coding

F1.5 is assigned to you, but three of its prerequisites belong to teammates and all three are still in **Backlog**:

Issue	Title	Owner	Status	What F1.5 needs from it
#13	F1.2 — Design the relational schema	Brian-Two	In Review	`users` table shape: `password_hash` , `role` , `email` uniqueness
#14	F1.3 — API service and Postgres + staging	oanyamele	Backlog	The process your endpoints live in; the database connection
#15	F1.4 — Repository interface	neaylasimone	Backlog	`UserRepository` , so `Login.tsx` stops importing `src/data/`

If you wait for all three, you do nothing for two weeks and then have five points due in three days. If you ignore them, you build a second competing server and the merge is a fight. **Neither.** The plan below is deliberately sequenced so Phase 0 and Phase 1 produce real, reviewable artifacts that depend on nothing, and the code that touches shared surfaces lands last.

Do this in the first 48 hours, before writing any code:

1. Get F1.2's ERD to include `users.password_hash TEXT NOT NULL` , `users.role` , `CREATE UNIQUE INDEX ON users (lower(email))` , and a `sessions` (or refresh-token) table. It is far cheaper to add these to Brian's schema review than to migrate later.
2. Agree with Ogechi (F1.3) on exactly three things and write them in the issue thread: the server framework, where auth route files live (propose `server/src/routes/auth.ts`), and who owns the session-secret environment variable.

3. Agree with Neayla (F1.4) on the `UserRepository` method signatures you need: `findByEmail`, `create`, `touchLastLogin`. Give her the signatures so the seam is shaped correctly the first time.
4. Tell Ogechi (F19, RBAC) the shape of the session claim your middleware will attach to the request, so his authorization checks read from your object instead of inventing one.

Record all four agreements as comments on issue #16. That thread *is* your evidence of cross-team coordination for the sprint review.

---

4. Decisions to make and write down

Do not start Phase 2 until these are settled. Put them in `docs/adr/0002-authentication.md` — the ADR directory is already required by F1.1.

Decision	Recommendation	Why
Hash algorithm	**argon2id** if the host supports native modules; otherwise **bcrypt** at cost 12	Both satisfy the criterion. Bcrypt is the safer choice for a student project on constrained hosting because it is pure-JS-friendly.
Session mechanism	**Opaque session ID in an httpOnly cookie**, row in a `sessions` table	The issue accepts "httpOnly cookies or short-lived JWTs," but the criterion also demands *revocation*. Revoking a JWT requires a denylist, which is a session table with extra steps. Skip the detour.
Cookie attributes	`httpOnly; Secure; SameSite=Lax; Path=/`, 7-day absolute expiry	`SameSite=Lax` works with the SPA on the same site. If staging ends up on a different origin than the client, you need `SameSite=None; Secure` **and** CORS `credentials: true` — settle this with Ogechi in §3 step 2.
Who can become an officer	Self-signup creates `student` **only**. Officer role is granted by an admin, or by the F17/F18 organization-verification flow.	Closes §2.3. Also lines up with US-08, which says an administrator approves org leaders before they can post.
Seed accounts	**Delete** `password123`. Provide a `server/scripts/seed-dev-users.ts` that reads passwords from env vars and refuses to run when `NODE_ENV=production`.	Satisfies "existing seed accounts migrated or reset" without breaking local demos.
Migration of existing plaintext users	Nobody real has an account. **Drop them.** Document that choice.	Writing a rehash-on-next-login migration for two fake accounts is wasted points. Say so in the ADR — "considered and rejected, with reason" scores better than silence.

---

5. Work plan

Five phases. Phase 0 and 1 are unblocked today.

Phase 0 — Groundwork (no dependencies) · ~0.5 pt

- [ ] Re-verify the evidence in §2 against the current `main` and update issue #16, including the `:179` → `:187` correction and the new §2.3 self-signup evidence.
- [ ] Write `docs/adr/0002-authentication.md` with the §4 decisions.
- [ ] Post the four coordination agreements from §3 as comments on issue #16.
- [ ] Open a draft PR early with the ADR only, so review starts before the code exists.

Phase 1 — The contract (no dependencies) · ~1 pt

Write the API contract and the zod schemas first. This is the piece that lets you work in parallel with F1.3 instead of behind it, and `zod` is already a dependency.

- [] `docs/api/auth.md` — endpoint table, request/response bodies, status codes, error codes, cookie behavior:

Method	Path	Purpose	Success	Failure
`POST`	`/api/auth/signup`	Create a **student** account	`201` + `Set-Cookie` + safe user	`409` email taken, `400` validation
`POST`	`/api/auth/login`	Exchange credentials for a session	`200` + `Set-Cookie` + safe user	`401` generic, `429` throttled
`POST`	`/api/auth/logout`	Revoke the current session	`204` + cookie cleared	`204` (idempotent)
`GET`	`/api/auth/me`	Who am I, per the server	`200` safe user	`401`

- [ ] Define the **safe user** DTO explicitly — `{ id, email, name, role, createdAt }`. No `password`, no `password_hash`, ever. Give it a name and a single serializer function so it cannot drift.
- [ ] `shared/schemas/auth.ts` — zod schemas for signup and login, exported for both the server (validation) and the client (form resolvers). Includes the university-email rule and a minimum password length.

Phase 2 — Server implementation · ~2 pts · needs F1.3's process + F1.2's schema

- [ ] Migration adding `password_hash`, `role`, the case-insensitive unique email index, and the `sessions` table. Coordinate with F1.2 rather than writing a competing one.
- [ ] `server/src/lib/password.ts` — `hash(plain)` and `verify(plain, hash)`. Two functions, one file. This is the only place in the codebase that touches a plaintext password.
- [ ] `server/src/lib/session.ts` — create, look up, revoke, and sweep expired sessions.
- [ ] `server/src/routes/auth.ts` — the four endpoints. Login **always** does a hash comparison even when the email is unknown (compare against a dummy hash) so response timing does not reveal account existence.
- [ ] `server/src/middleware/requireAuth.ts` and `requireRole(...roles)`. These attach the session claim and are what F19 builds on — hand them to Ogechi.
- [ ] In-memory rate limiter keyed on `email + IP`, roughly 5 attempts per 15 minutes, with a documented lockout. Note in the ADR that a single-process limiter is sufficient at this scale and what would replace it (Redis) if it were not.
- [ ] Audit log writes on signup, login success, login failure, logout, and role change — actor, action, timestamp, IP. Never the password or the hash.

Phase 3 — Client cutover · ~1 pt · needs F1.4's `UserRepository`

- [ ] `AuthContext` stops being the source of truth. Replace the `localStorage` read at `src/contexts/AuthContext.tsx:25` with a `GET /api/auth/me` query, and delete the write at `:32`. The `flyerrr_auth` key goes away entirely.
- [ ] `src/pages/Login.tsx` stops importing `authenticateUser` and `createUser` from `src/data/appDatabase` (line 9) and calls the repository instead. Remove the role argument from the signup path.
- [ ] Keep the role selector on the login screen as a *sign-in convenience* if the team wants it, but it must not be sent on signup. Or remove it — simpler, and the server knows the role anyway.
- [ ] Delete the `password` field from `AppUser` and the two `password123` seeds in `src/data/appDatabase.ts`. Any localStorage adapter that survives as the offline path stores the safe user DTO only.
- [ ] `ProtectedRoutes` in `src/App.tsx` renders a loading state while `/me` resolves, then redirects on `401`. State clearly in the PR description that this guard is now **UX only** and the real boundary is server-side.

Phase 4 — Proof · ~0.5 pt

The acceptance criteria say "verified by test." These are the tests that do it.

- [ ] `password.test.ts` — the same password hashes to different values twice; `verify` accepts the correct password and rejects a wrong one.
- [ ] `signup.test.ts` — duplicate email returns `409`; a response body never contains `password` or `password_hash`; **a signup request that includes `role: "org_officer"` produces a `student` account.** ← closes §2.3.
- [ ] `login.test.ts` — wrong password and unknown email return the *byte-identical* generic message; the 6th attempt in the window returns `429`.
- [ ] **The bypass test.** Set `flyerrr_auth` to `{"isLoggedIn":true,"role":"org_officer"}` with no valid cookie, then call the officer-only endpoint. Assert `401` / `403` and assert the database is unchanged. This single test is what the Definition of Done in `docs/BACKLOG_INTEGRATION.md` Issue 2 asks for — name it so a grader can find it.
- [ ] `logout.test.ts` — the session row is gone and the cookie no longer authenticates.
- [ ] `grep -rn "password" src/` shows no plaintext storage or comparison.
- [ ] `npx tsc --noEmit -p tsconfig.app.json`, `npm run lint`, `npm test`, and `npm run build` all pass. Add the auth tests to `.github/workflows/ci.yml` (F3.1).

---

6. Acceptance criteria — copy back into issue #16

The first six are the current criteria. Three are new, and you should add them.

- [ ] Passwords hashed with bcrypt or argon2, server-side only
- [ ] No credential is ever written to client storage
- [ ] Session tokens are httpOnly cookies or short-lived JWTs
- [ ] Every privileged endpoint enforces role server-side
- [ ] Editing localStorage cannot grant officer access — verified by test
- [ ] Existing seed accounts migrated or reset

- [ ] **New —** A signup request cannot set its own role; self-registration always yields `student`, verified by test
- [ ] **New —** Login failures return an identical generic message for unknown-email and wrong-password, and are rate-limited
- [ ] **New —** `docs/adr/0002-authentication.md` records the hash algorithm, session mechanism, and cookie policy with at least two alternatives considered

7. Definition of done

A teammate on a clean checkout can sign up, sign in, and reach the feed; the `users` table shows an argon2/bcrypt hash and no plaintext anywhere; and an automated test proves a user who tampers with client state cannot reach any officer-only screen or endpoint.

---

8. Suggested schedule against Sprint 1 (Weeks 2–4)

When	What	Blocked by
Week 2, days 1–2	Phase 0 — evidence refresh, ADR, four coordination comments	nothing
Week 2, days 3–5	Phase 1 — API contract + shared zod schemas; draft PR open	nothing
Week 3	Phase 2 — server implementation against F1.3's process	F1.3 process reachable, F1.2 migration merged
Week 4, days 1–3	Phase 3 — client cutover	F1.4 `UserRepository` merged
Week 4, days 4–5	Phase 4 — tests, CI wiring, PR review	—

If F1.3 slips past Week 3: do not stall. Stand the same four endpoints up against an in-memory store behind the repository interface, with the real hashing and the real cookie handling. Swap the store for Postgres when F1.3 lands. The hashing, session, rate-limit, and bypass-test work — which is the actual substance of F1.5 — is fully testable without a database, and the swap touches one file. Say this in the sprint review; "I de-risked a dependency I did not control" is the right story to be able to tell.

9. Explicitly out of scope

Name these in the PR so a reviewer does not ask for them:

- The second authentication factor — that is F07-03 in the working doc, a separate issue
- Password reset and email verification — needs F22 (notification service, unassigned)
- The full role/permission matrix and its enforcement everywhere — that is F19, Ogechi's
- OAuth or Howard SSO — not in the backlog; mention as future work in the ADR
- Geofencing, check-in tokens, attendance — F2.x, downstream of this